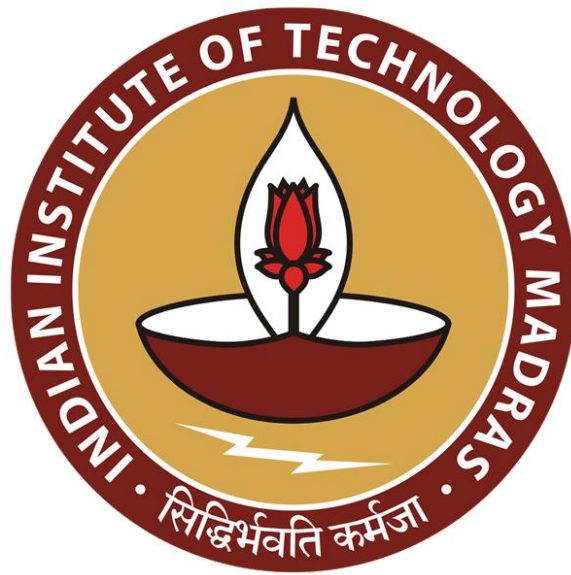




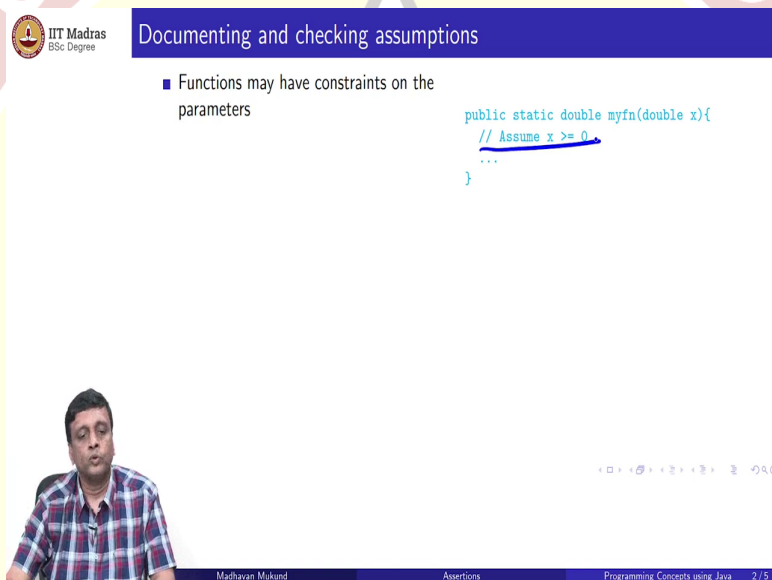
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Assertions

So, we began this week's lectures by talking about errors and exceptions. And we digressed a little bit by talking about packages, which is an organizational concept for code. It is not directly related to exceptions error. So, let us come back to the theme of exceptions and errors. Let us talk about something called assertions.

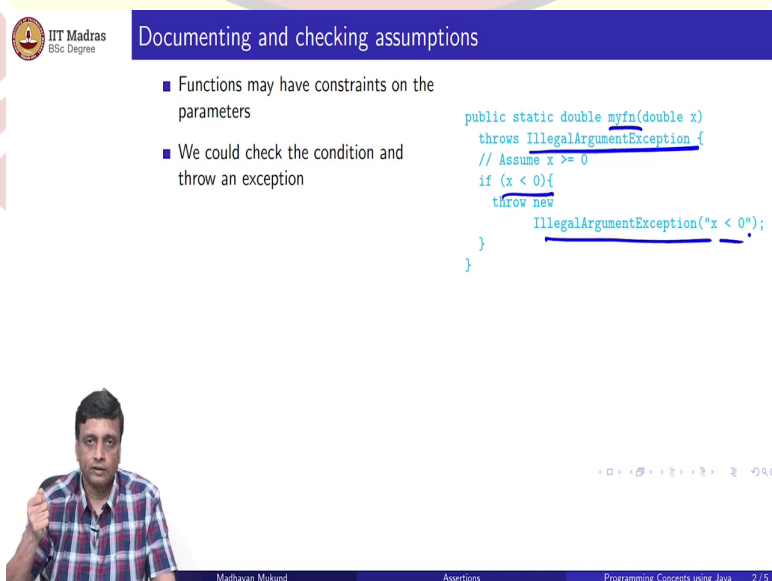
(Refer Slide Time: 0:33)



The slide is titled "Documenting and checking assumptions". It features the IIT Madras logo and text in the top left corner. A bullet point states: "Functions may have constraints on the parameters". To the right, a code snippet is shown:

```
public static double myfn(double x){  
    // Assume x >= 0  
    ...  
}
```

 The speaker, Professor Madhavan Mukund, is visible in a small video window at the bottom left. The bottom of the slide has a navigation bar with the text "Madhavan Mukund", "Assertions", "Programming Concepts using Java", and "2/5".



The slide is titled "Documenting and checking assumptions". It features the IIT Madras logo and text in the top left corner. Two bullet points are listed: "Functions may have constraints on the parameters" and "We could check the condition and throw an exception". To the right, a code snippet is shown:

```
public static double myfn(double x)  
    throws IllegalArgumentException {  
    // Assume x >= 0  
    if (x < 0){  
        throw new  
            IllegalArgumentException("x < 0");  
    }  
}
```

 The speaker, Professor Madhavan Mukund, is visible in a small video window at the bottom left. The bottom of the slide has a navigation bar with the text "Madhavan Mukund", "Assertions", "Programming Concepts using Java", and "2/5".

So, when we write code, as we said before, sometimes there are some implicit constraints, like we talked about this linear list of order numbers, which is not allowed to take negative inputs. Similarly, we might write something like a square root function, which does not like negative inputs. So, in our comments, we might have a statement like this, now we are assuming that x is positive.

Now, we have seen that exceptions give us a way to validate this, so we can check that the arguments to a function obey the constraints that we want. So, one way to do that would be to create, for instance, some kind of exception and throw it. So, we can say that this function, my function now throws an illegal argument exception. And when there is an illegal argument, exception triggered, if I find that x is negative, then I will throw this new exception which say an error message saying x is less than 0. So, this is the way that we have seen, which allows us to enforce an assumption or a constraint using an exception.

(Refer Slide Time: 1:42)

Documenting and checking assumptions

- Functions may have constraints on the parameters
- We could check the condition and throw an exception
- What if `myfn` is only used internally by our own code
 - Flag errors during development, debugging
 - But diagnostic code should not trigger at run time
 - Performance, and other considerations

```
public static double myfn(double x)
throws IllegalArgumentException {
    // Assume x >= 0
    if (x < 0) {
        throw new
            IllegalArgumentException("x < 0");
    }
}
```

Madhavan Mukund Assertions Programming Concepts using Java 2/5

Now, this might be reasonable if my function is called from a variety of contexts. So, it is a public function, which is called by a variety of contexts. And I have advertised publicly that this function should not be called with a negative argument. So, anybody who calls this function with a negative argument should be told that they have done something wrong. So in such situations, throwing an exception is quite reasonable.

But it could also be that this function is something which I am using as part of a larger scheme of things. So, it is actually internal to my code development, I am writing a lot of

code and one of the things I need is this function. And I know that this function will never be called with a negative number. So, I have the way to set up this function is supposed to be called only with positive numbers.

But I would like to make sure while developing the code that this is not actually the cause of an error. So, I would like to document that. So, if I now keep this code in, it will serve the same purpose. So, whenever I call this code from within my own development code, it will again throw an error and I can check. So, wherever I call it, I can check it.

But now what happens therefore is that this diagnostic code is now a part of my main code, and it will run forever unless I remove it. So, if it remains in my production code, it will not create any problems. Because if there are no errors, then of course, it will run because if this if I never call it takes less than 0, this exception never be triggered.

But it creates other problems in that everybody who calls this has to know this, but assuming nobody else is going to call it, it still means that every time this function is called this check, check remains explicitly executed. And there may be many such checks. So, you might have many functions, which have these kinds of checks, which are purely internal checks within my code development.

So, overall I have a performance issue. I am making a lot of I have, I needed these checks when I was debugging the code and building it, I should not have these checks active when I am running the code, after it has been debugged and deployed. So, this is one of the things that we would like to deal with.

(Refer Slide Time: 3:47)

Documenting and checking assumptions

- Functions may have constraints on the parameters
- We could check the condition and throw an exception
- What if `myfn` is only used internally by our own code
 - Flag errors during development, debugging
 - But diagnostic code should not trigger at run time
 - Performance, and other considerations
- Instead, "assert" the property you assume to hold

```
public static double myfn(double x){
    assert x >= 0;
}
```

X $x < 0$
✓ $x \geq 0$



Madhavan Mukund

Assertions

Programming Concepts using Java 2/5

So, the option that we are going to propose is something called an assertion. So, there is a statement that Java has and other programming languages also have, it is called assert. So, what you say is that less like you have an if condition and a while condition, you say assert, and then you make a Boolean expression, expressing some property that you would say should be true.

So, in this case, we do not want, so we do not want x to be less than 0. So, we do want x to be bigger than or equal to 0. So, the negation of what we do not want to see. So, we are asserting the positive statement. This is the assumption the assumption is that x is not negative. So, this now is what we are going to talk about this assert statement.

(Refer Slide Time: 4:28)

Assertions

- If assertion fails, code throws `AssertionError`

```
public static double myfn(double x){
    assert x >= 0;
}
```

False



Madhavan Mukund

Assertions

Programming Concepts using Java 3/5

- If assertion fails, code throws `AssertionError`
- This should **not** be caught
 - Abort and print diagnostic information (stack trace)
- Can provide additional information to be printed with diagnostic message

```
public static double myfn(double x){
    assert x >= 0 : x;
}
```



Madhavan Mukund

Assertions

Programming Concepts using Java 4/5

So, what happens if at some point, we call this function and it turns out that x is negative, then this will become false. So, if the assertion expression turns out to be false, then this generates a type of error called an assertion error. So, this is an error in the same way of any other Java error. So, it belongs in that error hierarchy.

But the point is that assertions are not as we said, to be confused with exceptions. So, when we throw exceptions, we do expect the calling code to take evasive action or do something with it. So, that has a different purpose. So, there the exceptions are thrown with the purpose, if the exception is thrown with the intention that the calling code can take some action to guard against the error that has been generated.

In this case, an assertion error is really something which should not happen. So, we should flag it and say this has happened and then we should go and investigate the code as to why it has happened. It is not something that I want to fix at runtime. So, even though it is an error, you should not catch it. So, this is a kind of coding convention, assertion error should not be caught, they should just let the code abort.

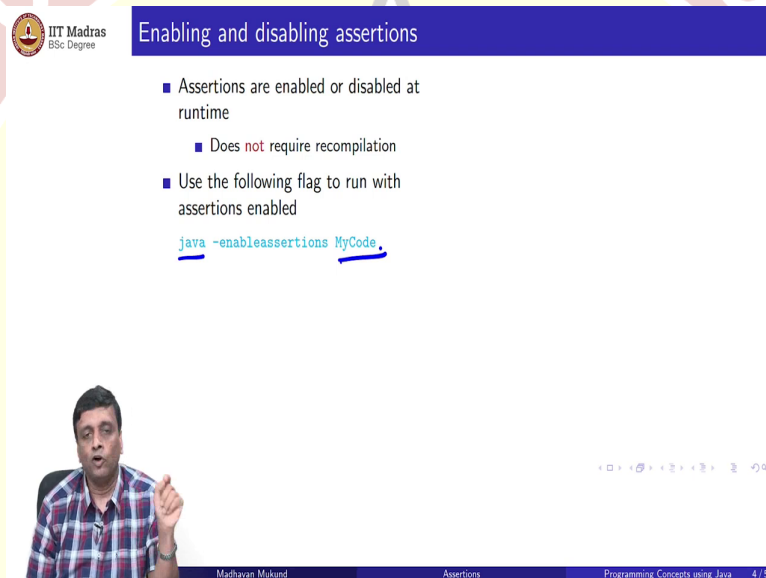
And then when it aborts, it will print out various things about what happened. It is sometimes called a stack trace, that is all the memory contents that have been and then you have to figure out from the documentation how to interpret what exactly happened. So, the idea with an assertion error is it is a fatal error, you should not try to catch it and continue.

So, you can also provide a little bit of extra information instead of just saying, assertion failed. At this point, you can pass the value that, for instance, created the assertion failed. So,

the way you do it is after the assertion statement, you put a colon and then you put the thing that you need to pass on. So, this will be printed additionally, as a diagnostic method. So, that is the only thing you are supposed to do with assertions other than letting the code crash.

You provide a predicate that is a function which evaluates to true or false a Boolean expression. If it values to true, nothing happens, you just proceed, if it evaluates to false, false, it will throw an assertion error, you should just let the program crash, not catch it. And in addition, you can provide information which will give you a diagnostic string at the time the assertion error is thrown.

(Refer Slide Time: 6:48)



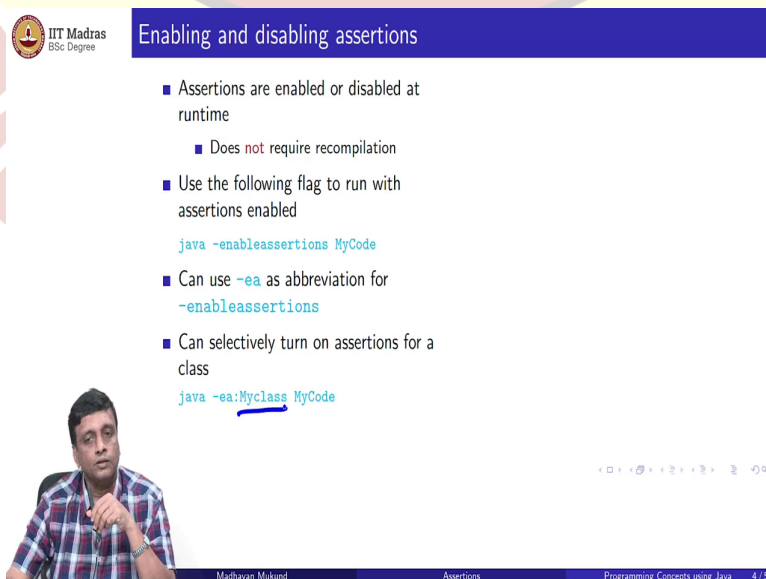
IIT Madras
BSc Degree

Enabling and disabling assertions

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled

```
java -enableassertions MyCode.
```

Madhavan Mukund



IIT Madras
BSc Degree

Enabling and disabling assertions

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled

```
java -enableassertions MyCode
```
- Can use **-ea** as abbreviation for **-enableassertions**
- Can selectively turn on assertions for a class

```
java -ea:MyClass MyCode
```

Madhavan Mukund

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled


```
java -enableassertions MyCode
```
- Can use `-ea` as abbreviation for `-enableassertions`
- Can selectively turn on assertions for a class


```
java -ea:Myclass MyCode
```
- ...or a package


```
java -ea:in.ac.iitm.onlinedegree MyCode
```



Madhavan Mukund

Assertions

Programming Concepts using Java 4/5

So, one of the things about assertions is we said when you put an exception and you leave it in your code, that if statement will be executed each time. So, that is a kind of bottleneck in terms of performance, because you cannot turn it off. Or you have to go through of course and clean up your code and remove all those diagnostic exceptions that you generate.

So, you have two options. And if you leave any of them in, then that performance is going to be held because it is going to keep doing. Assertions, on the other hand, are statements that you can leave in your code, because you can actually tell Java, whether you want it to check assertions or not check assertion.

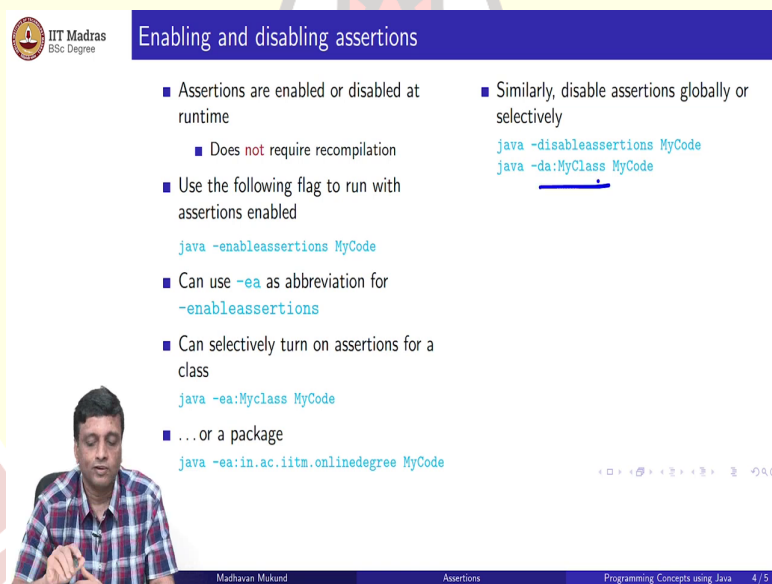
So, checking assertions is something which happens not through the compiler, but it happens at runtime. So, there, you can tell Java when it runs your code, check assertions, or do not check assertions. So, this does not require re compilation or editing the code. So, if I want to tell it to check assertions, then when I so remember that this is the Java runtime, so it is not the compiler, but it is the one which takes a compiled class file, and transit, you can give it the option, enable assertions. And this will mean that every assertion which occurs in your code will actually be checked.

By default, it will not be checked. And you can abbreviate this enable assertions by just `ea`. So, this is a way of running your code with assertions enabled. Now, you need not enable all assertions. So, you can turn on assertions only for particular pieces of your code, which are new and which you are trying to test out. So, for instance, if, if this, `Myclass` is something which you are suspicious about, you can say enable assertions only for `Myclass`.

You can also do this for an entire package, you can say that I added this package so please check it for all the code that is there in my package, do not worry about other packages. So, you can control first of all, whether assertions are checked at all. And you can also control where all these are checked. They can be in specific classes, they can be in packages, and so on. And you can combine, you can say, do it in this class and in this package.

So, you can put more than one minus ea in that command line. So, this is the advantage of using assertions as opposed to the exception route in the exception route, you cannot turn on and turn off exceptions, they will always be part of if statements and they will be checked. Whereas here, you can actually say, at runtime, the assertions remain in your code. But whether or not they are actually executed and validated, is something that you can decide at runtime.

(Refer Slide Time: 9:17)



IIT Madras
BSc Degree

Enabling and disabling assertions

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled

```
java -enableassertions MyCode
```
- Can use **-ea** as abbreviation for **-enableassertions**
- Can selectively turn on assertions for a class

```
java -ea:MyClass MyCode
```
- ... or a package

```
java -ea:in.ac.iitm.onlinedegree MyCode
```
- Similarly, disable assertions globally or selectively

```
java -disableassertions MyCode
java -da:MyClass MyCode
```

Madhavan Mukund Assertions Programming Concepts using Java 4/5

Enabling and disabling assertions

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled


```
java -enableassertions MyCode
```
- Can use **-ea** as abbreviation for **-enableassertions**
- Can selectively turn on assertions for a class


```
java -ea:MyClass MyCode
```
- ...or a package


```
java -ea:in.ac.iitm.onlinedegree MyCode
```
- Similarly, disable assertions globally or selectively


```
java -disableassertions MyCode
java -da:MyClass MyCode
```
- Can combine the two


```
java -ea in.ac.iitm.onlinedegree
    -da:MyClass MyCode
```



Madhavan Mukund

Assertions

Programming Concepts using Java 4/5

Enabling and disabling assertions

- Assertions are enabled or disabled at runtime
 - Does **not** require recompilation
- Use the following flag to run with assertions enabled


```
java -enableassertions MyCode
```
- Can use **-ea** as abbreviation for **-enableassertions**
- Can selectively turn on assertions for a class


```
java -ea:MyClass MyCode
```
- ...or a package


```
java -ea:in.ac.iitm.onlinedegree MyCode
```
- Similarly, disable assertions globally or selectively


```
java -disableassertions MyCode
java -da:MyClass MyCode
```
- Can combine the two


```
java -ea in.ac.iitm.onlinedegree
    -da:MyClass MyCode
```
- Separate switch to enable assertions for system classes


```
java -enablesystemassertions MyCode
java -esa MyCode
```



Madhavan Mukund

Assertions

Programming Concepts using Java 4/5

So, the converse is also true, you can disable assertions. So, you can say minus disabled, or minus. So, in particular, you can disable assertions again for a selective class. In particular, it might make sense because you might say that I want to enable assertions for this entire package, but I want to disable it for a particular class in that package. So, that is also helpful. So, you might want to enable it for everything except some classes which you know are fine and so on.

What about the classes which are somehow built into Java? So, these are all classes. So, Java has something we are not really discussed very much in detail about how this works, but what Java does is it has something called a class loader as it as an when, remember that Java compiler will recursively look for all the classes which are used and compile them. So, you

only need to compile the first file, which has the main everything which it needs will recompile automatically.

Correspondingly, all the classes that are needed will be loaded at runtime by something called a class loader. But there are some built in classes which are part of the JVM, they never get loaded at runtime. And they are already there. So, those things are not affected by this enable assertions, disable assertions. So, they are not part of this thing at all. This only refers to classes, which you have defined and which are kind of called by you by defined by other people's are not the system things. So, if you want to enable assertions on the system thing, you have to use a different flag, which is called `enableSystemAssertions` or `esa`, separate switch for system assertions.

(Refer Slide Time: 10:47)

Summary

- Assertion checks are supposed to flag fatal, unrecoverable errors
 - Do not **catch** them!
- If you need to flag the error and take corrective action, use exceptions instead
- Turned on **only** during development and testing
 - Not checked at run time after deployment

Madhavan Mukund Assertions Programming Concepts using Java 5 / 5

So, to summarize, if you are just trying to do some sanity check in your code for the for development, and it is not something which needs to be flagged to users of your code, because it is not a function that other people are going to use for whom the error message is useful, then it is cheaper and simpler to use asserts rather than exceptions.

So, an assertion error will flag something which is unrecoverable. So, even though it sits in this error hierarchy of Java do not catch it, and assertion errors indicating that something went wrong with your code. So, you have to go and fix it. It is not something that you should dynamically deal with as you are going along, like normal errors and exceptions.

So, if you actually need to do something at runtime, and flag the error and do something about it, then go back to exceptions, by all means. But if you are using it only for debugging, then use assertions. And the main fact about assertions is that typically only used during testing. So, when you are developing the code and testing it, then you will use assertions. So, it is not checked at runtime. So, at runtime, you basically throw this code out and you do not take the assertions because you believe that you have tested adequately so the assertions online.

So, it is a bit like saying that, you know, while you are training to swim, you wear a life jacket, but then when you go out and swim in a competition, you are not going to wear a life jacket, you are going to run unconstrained you are going to swim unconstrained. So, it is like that. So, these are kind of constraints, which prevent your code from doing strange things without your knowing about it when you are developing it. But afterwards, when you actually are confident that this thing will work as it is intended. You throw away these constraints and let it run without these constraints. So, that is how assertions work.

